TD Chapitre 5

EXERCICE 1

Créer une application permettant d'annoncer en ligne des voitures à vendre. L'application aura les acteurs :

1. **vendeur** avec ses cas d'utilisations :
   (a) ouvrir un compte (s'inscrire). Le compte créé est par défaut inactif.
   (b) publier, modifier et supprimer une annonce
   (c) lister ses annonces.
2. **admin** avec ses cas d'utilisations :
   (a) activer, désactiver et supprimer le compte d'un vendeur,
   (b) lister les vendeurs.
3. **visiteur** avec ses cas d'utilisations :
   (a) voir la liste des annonces,
   (b) voir les détails d'une annonce.

Les acteurs **admin** et **vendeur** doivent s'authentifier avant d'accéder à l'application. la durée de leurs sessions respectives et de 5 et 10 minutes.

Indications : Créer un projet Spring Boot avec Spring initializr sous Netbeans
1- Choisir les dépendances :

```
Spring Boot DevTools
Spring Web
Spring Data JPA
MySQL Driver
Spring security
```

2- Dépendances

```
<dependency>
  <groupId>org.eclipse.jetty</groupId>
  <artifactId>glassfish-jstl</artifactId>
  <version>11.0.19</version>
</dependency>
<dependency>
  <groupId>org.apache.tomcat.embed</groupId>
  <artifactId>tomcat-embed-jasper</artifactId>
  <scope>provided</scope>
</dependency>
```

3- Configurer la prise en compte de JSP dans application.properties.

```
spring.mvc.view.prefix=/WEB-INF/jsp/
spring.mvc.view.suffix=.jsp
```

4- Créer les répertoires

```
src/main/webapp/WEB-INF/jsp
src/main/resources/static/images
src/main/resources/static/css
src/main/resources/static/js
```

5- Configurer la source de données dans application.properties après avoir créé la base de donnée td5exo1

```
spring.datasource.url=jdbc:mysql://localhost:3306/td5exo1
spring.datasource.username=user
spring.datasource.password=mot_de_passe
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.properties.hibernate.format_sql=true
```

6- Création de l'entité User

```
Créer l'entité User dans son paquet user. Décocher le checkbox "Create Persistence Unit".
La classe User implémente les interfaces Serializable et UserDetails
Ajouter l'annotation @Table(name="appuser")
Créer l'interface UserRepository dans le paquet user avec :
   Base Interface : JpaRepository, Entity Class : User, Id Class : Long.
Créer la classe service UserService dans le paquet user.
La classe service implémente UserDetailsService.
La classe service utilise un userRepository.
La classe service sera utilisée dans un contrôleur.
```

7- Configuration de la sécurité

```
Créer la classe de configuration de sécurité SecConfiguration.
@Configuration
@EnableWebSecurity
public class SecConfiguration {
  private final UserService userService;

  public AppSecurityConfiguration(UserService userService) {
         this.userService = userService;
  }

  @Bean
  public SecurityFilterChain filterChain(HttpSecurity httpSecurity) throws Exception {
    return httpSecurity
      .csrf(csrf->csrf.disable() )
      .build();
  }

  public void configureGlobal(AuthenticationManagerBuilder auth) throws Exception {
    auth
      .userDetailsService(userService);
  }

  @Bean
   public static PasswordEncoder passwordEncoder() {
      //return new BCryptPasswordEncoder();
      return NoOpPasswordEncoder.getInstance();
   }

   public void configureGlobal(AuthenticationManagerBuilder auth) throws Exception {
      auth
            .userDetailsService(userService);
            //.passwordEncoder(passwordEncoder());
   }
}
```

8- Autorisations

```
.authorizeHttpRequests((authorize) -> authorize
.dispatcherTypeMatchers(DispatcherType.FORWARD).permitAll()
.requestMatchers("/images/**").permitAll()
.requestMatchers("/voir").hasRole("SUPERUSER")
```

9- Personnaliser le formulaire de login

```
.formLogin(form -> form
.loginPage("/login")
.permitAll()
)
.logout(logout -> logout
.permitAll()
)
```